课 程 内 容

第 1 章 编 译 引 论

第 2 章 形式语言与自动机基础

第 3 章 词法分析 (Lexical Analysis)

第 4 章 语法分析(Syntax Analysis)
—— 自上而下分析法

第 5 章 语法分析(Syntax Analysis)
—— 自下而上分析法

第 6 章 运 行 环 境

第 7 章 语义分析与中间代码生成

第 8 章 代码优化（optimization）

第 3 章　词法分析 (Lexical Analysis)

3.1 词法分析程序的功能

3.2 词法分析器的设计与实现

3.3 词法分析器的自动生成

词法分析程序的功能

完成词法分析任务的程序称为词法分析程序或词法分析器或扫描器。

从左至右地对源程序进行扫描，按照语言的词法规则识别各类单词，并产生相应单词的属性字。

- 关于词法分析程序

* 组织输入、扫描、分析、输出；

* 接收字符串形式的源程序，按照源程序输入的次序依次扫描源程序，在扫描的同时根据语言的词法规则识别出具有独立意义的单词，并产生与源程序等价的属性字（Token）流 .

- 关于词法分析程序 （作为一个独立子程序）

(1) 只要不修改接口，则词法分析器所作的修改不会影响整个编译器，且词法分析器易于维护；

(2) 整个编译器结构简捷、清晰；

(3) 可以采用有效的方法和工具进行处理。

例3.1　有如下C源程序片段

```
{
    int int1 ;

    int1 = 33 ;

    printf ( "int1= %d\n" , int1 ) ;
}
```

说明：

1. 词法分析是编译程序的第一个阶段且是必要阶段；

2. 词法分析的核心任务是扫描、识别单词且对识别出的单词给出定性、定长的处理；

3. 实现词法分析程序的常用途径：

   - 自动生成
   - 手工生成

第 3 章　词法分析 (Lexical Analysis)

3.1 词法分析程序的功能

3.2 词法分析器的设计与实现

3.3 词法分析器的自动生成

第 3 章　词法分析 (Lexical Analysis)

3.2 词法分析器的设计与实现

3.2.1　单词与属性字

3.2.2　源程序的输入与预处理

3.2.5　扫描器设计、实现技术

1. 单词

单词是语言中具有独立意义的最小语法单位。

要素 { 独立的意义
最小的语法单位

* 词法规则制约

例如，

…A*B…，单词是“A”、“*”和“B”。

…int int1…,单词是“int”和“int1”。

…A++*B…, 单词是“A”、“++”、“*”和“B”。

- 流行语言词法规则的表示：

3型文法 { BNF, EBNF }

正规式

<标识符> → a<后缀> | b<后缀> |…| z<后缀>

<后缀> → a<后缀> | b<后缀> |…| z<后缀> | 0<后缀>

|1<后缀> |…| 9<后缀>| a | …| z | 0 | … | 9

<标识符> → (a | …| z) { a | …| z | 0 | … | 9 }

标识符= (a | …| z) (a | …| z | 0 | … | 9) *

常用的程序设计语言的单词类：

（1） 关键字(亦称保留字，基本字等)

关键字一般是语言系统本身定义的，通常是由字母组成的字符串。

例如： C语言中的 **int, for, break, static, char, switch, unsigned**等， 关键字一般关联到语句的性质。

（2） 标识符

用来表示各类名字的标识。如，变量名，数组名，结构名，函数名，文件名等。

（3）常量

语言中各种类型的常数。如整型常数，实型常数，不同进制的常数，布尔常数，字符及字符串常数等。

- I常数：-26，19，0x123，037 …
- R常数：-1.6，2e-6，2.5e06，…
- B常数：TRUE，FALSE，0或非0，…
- Char、String：'$'，'\123'，“abc”，…

…

（4）运算符

表示程序中算术运算，逻辑运算，字符及位串等运算的确定字符(或串)。

例如，各类语言较通用的＋，－，*，/，**，
＜＝，＞＝，＜，＞等。
还有一些语言特有的运算符，如C语言中的++，？:，&，％＝等。
FORTRAN 语言中的 .AND.，.NOT.，.OR.。

（5）界限符

如逗号，分号，括号，单双引号等。

注意：

1） 同一个字符开头+后续字符⇨跨多个单词类；

— { – 2.6
– – j
–>
a –b
– = }

0 { 0.0
0x8a
037 }

2） 非单词成分和预处理成分；

例如，源程序注释；

预处理指令：# define…

include…

2. 属性字

对所识别的单词的数据结构表示。

L1= （T，C）

属性字

Token	Code

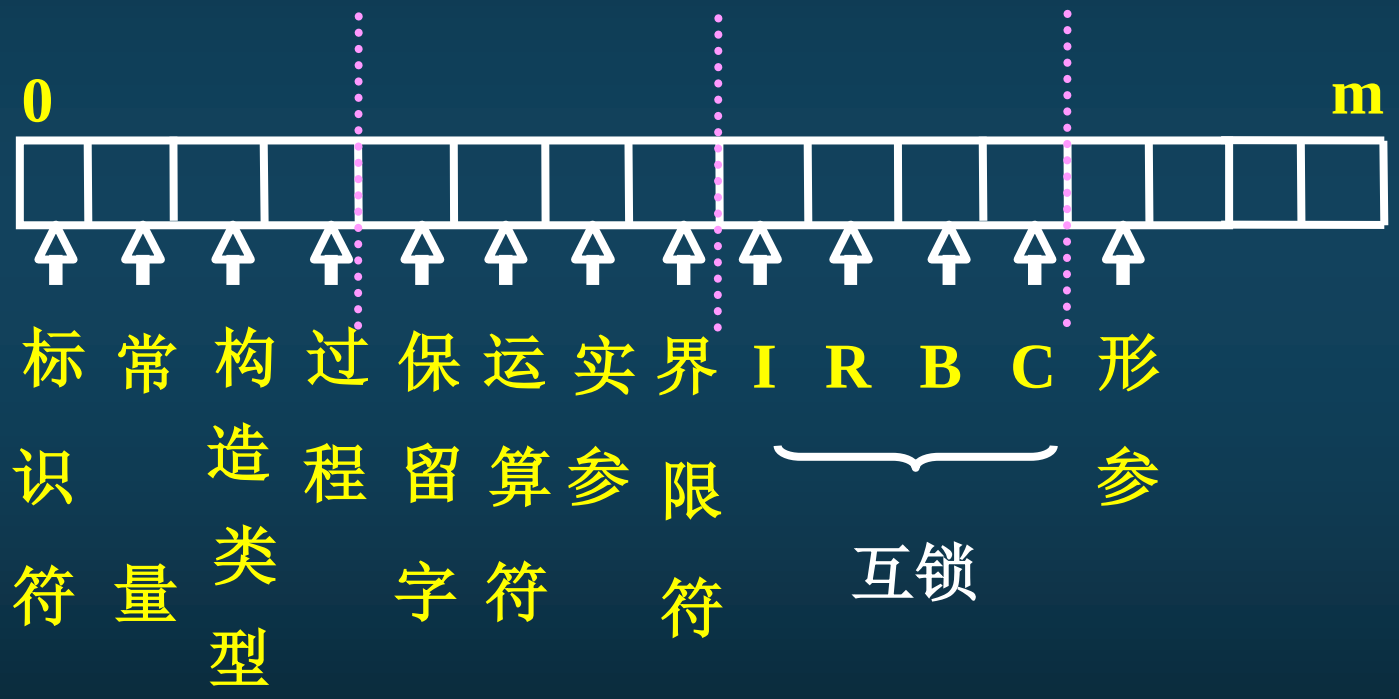
刻画单词类别（单词性质）

例如，标识符；运算符；…

单词的内码值（可空）

例3.2： 字长m=16的单词类别的设计。

考虑类PASCAL语言。



设有变量　real　x,a;　integer　b;

对语句　x= a+b;　词法分析结果：

	Token	Code	
x	8000	x	m=1000 0000 0000 0000
=	0400	=	m=0000 0100 0000 0000
a	8000	a	m=1000 0000 0000 0000
+	0400	+	m=0000 0100 0000 0000
b	8000	b	m=1000 0000 0000 0000
;	0100	;	m=0000 0001 0000 0000

L1

设有变量 real x,a; integer b;

对语句 x= a+b; 语法语义分析后：

	Token	Code	
x	8040	x	m=1000 0000 0100 0000
=	0400	=	m=0000 0100 0000 0000
a	8040	a	m=1000 0000 0100 0000
+	0400	+	m=0000 0100 0000 0000
b	8080	b	m=1000 0000 1000 0000
;	0100	;	m=0000 0001 0000 0000

✌ 注意：

关于属性字单词类别部分设计：

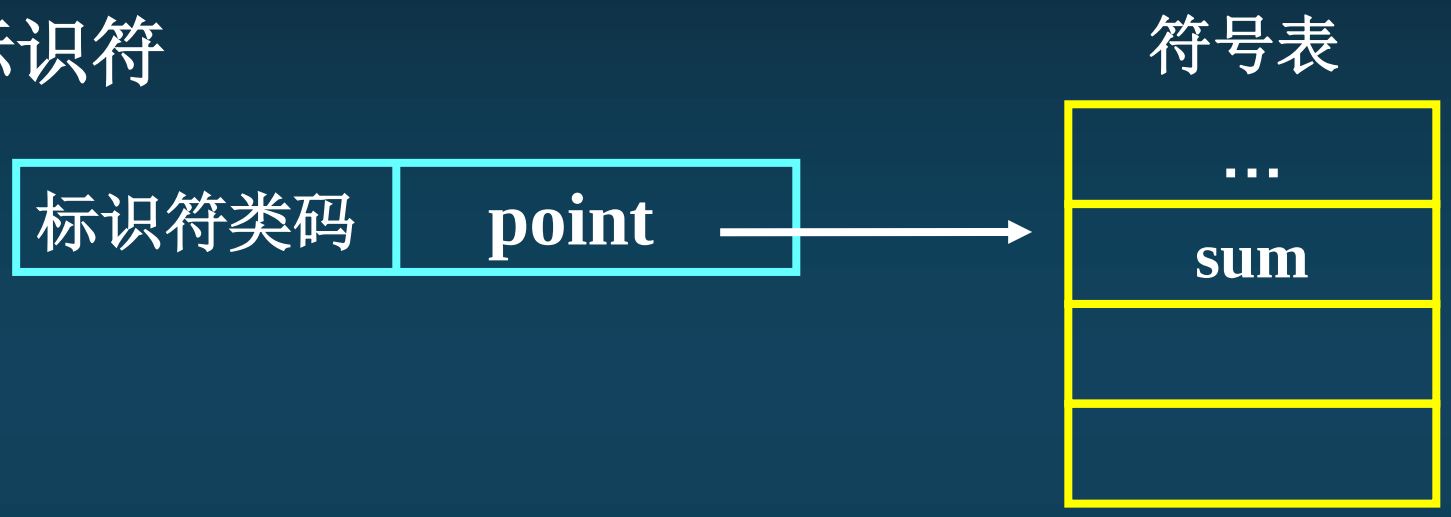
单词如何分 类？分为几类？怎样编码？单词类别部分包含 多少信息？

视具体情况而定；

考虑后续处理方便 。

常见类单词的属性字设计：

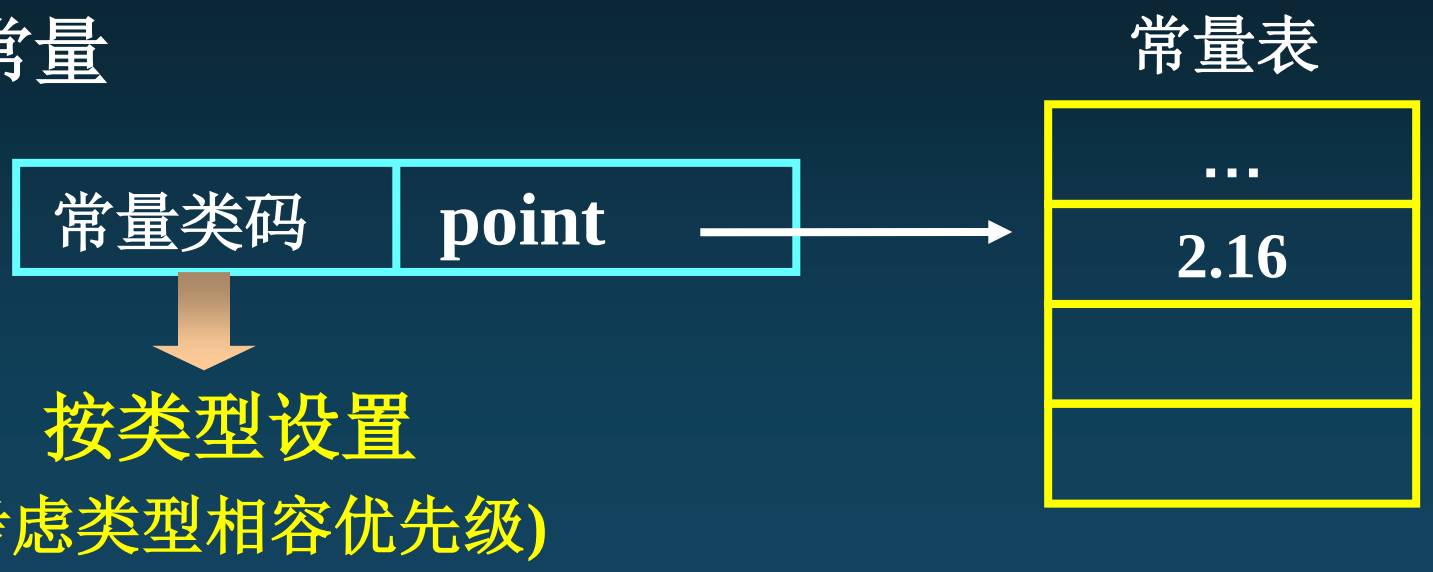
1. 标识符



2. 关键字

语言的关键字个数一般是固定的，考虑每个关键字对应一个属性类别码，内码值部分为空。

3. 常量



4. 运算符和界限符

1）按照一个符号分别对应一个属性类别码，内码值部分为空。

2）运算符可以考虑优先级相同的对应一个属性类别码。优先级高的类别编码值大。

例如

31	+
31	–
32	*
32	/
10	
…	

: ?

第 3 章　词法分析 (Lexical Analysis)

3.2 词法分析器的设计与实现

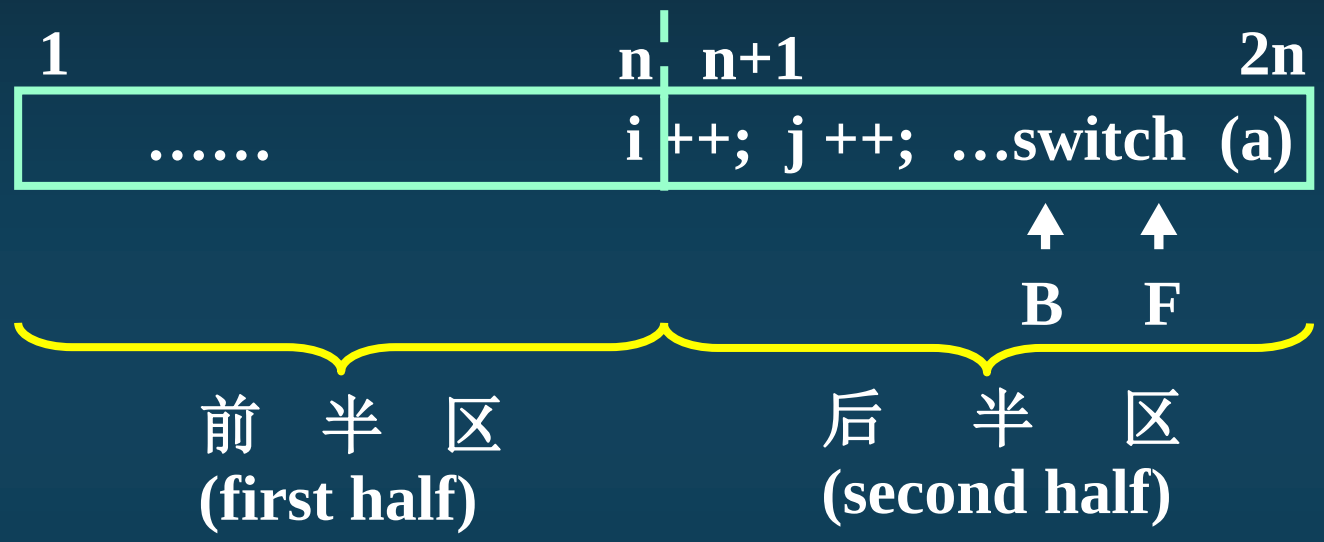
3.2.1　单词与属性字

3.2.2　源程序的输入与预处理

3.2.5　扫描器设计、实现技术

1. 输入缓冲区

成对且对半互补的输入缓冲区模式。



- **n:** 取2的整次幂；
- 每个半区的末尾设置标志“ eof ”表示读入该半区的源程序的结束；
- **B:** 单词w开始指针； **F:** 扫描w的指针；

- 两半区互补功能算法：

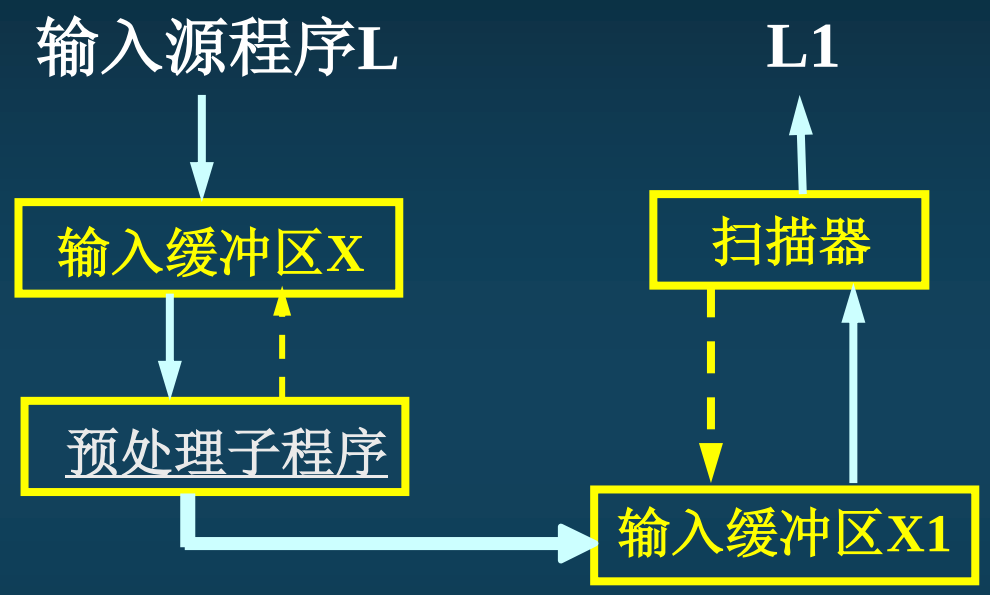
```
if ( F= 前 " eof ") { 重新分配、装入后半区;F++ };
    else if ( F= 后 " eof ")
                {重新分配、装入前半区；F=1};
            else F++;
```

另一个算法：

- 两半区互补功能算法：

```
if(F="eof"){
    if ( F= 前 " eof ") { 重新分配、装入后半区;F++ };
    else if ( F= 后 " eof ")
                {重新分配、装入前半区；F=1}；}
else F++;
```

2. 两个缓冲区的输入模式



控制线 数据线

X : 固定长度的存储空间；

预处理程序：（作用）

1） 减少内存空间占用；

2） 减轻扫描器实质性处理的负担；

预处理程序主要任务：

1） 滤掉源程序中的非单词成分(如无用空格；换行符等)；

2） 其他的预处理工作
- 滤掉注释；
- 宏替换；
- 文件包含的嵌入；
- 条件编译的嵌入；

第 3 章　词法分析 (Lexical Analysis)

3.2 词法分析器的设计与实现

3.2.1　单词与属性字

3.2.2　源程序的输入与预处理

3.2.5　扫描器设计、实现技术

- 设计工具 —— FA

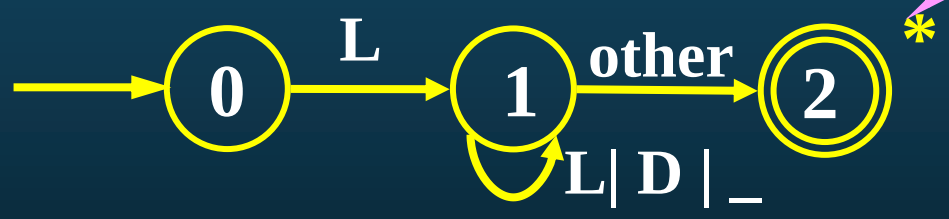
作为扫描器的状态转换图的构造：

step1：对语言的各类单词分别构造状态图；

例3.3　设某语言由标识符和无符号正整数两类单词构成，并设L表示字母，D表示十进制数字，又标识符和无符号正整数的正规式如下：

标识符：L(L|D|_)*

无符号正整数：DD*



到达此终态时多读了一个字符

其中：other表示非L| D | _字符

其中other表示非 D的字符

- 设计工具 —— FA

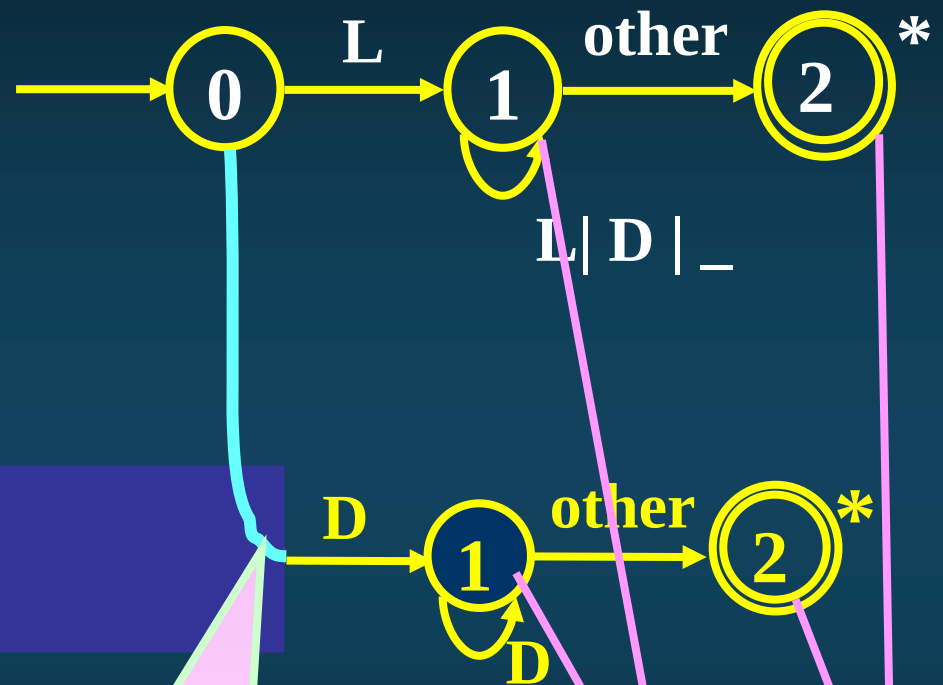
作为扫描器的状态转换图的构造：

step1： 对语言的各类单词分别构造状态图；

step2： 将各类状态图合并，构成一个能识别该语言所有单词的状态图。

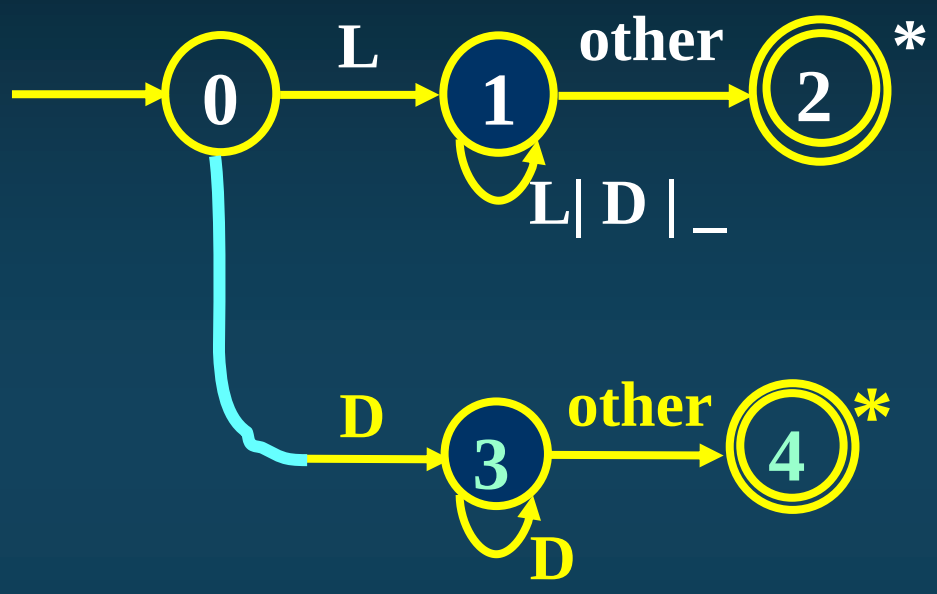
1）将各类单词的状态图的初态合并为一个惟一初态；

2） 调整冲突编号。



合并为一个识别该语言所有单词的NFA

调整冲突编号


0
L
1
other
2
*
L| D | _
D
3
other
4
*
D

例3.4　设C语言子集由下列单词符号构成，以正规式的形式表示：

关键字：**int, if, for**

标识符：　字母(字母|数字)*

无符号整常数：　数字(数字)*

运算符或分界符：　=，*, +，++，+=，{，}

标识符、关键字:

无符号整常数:

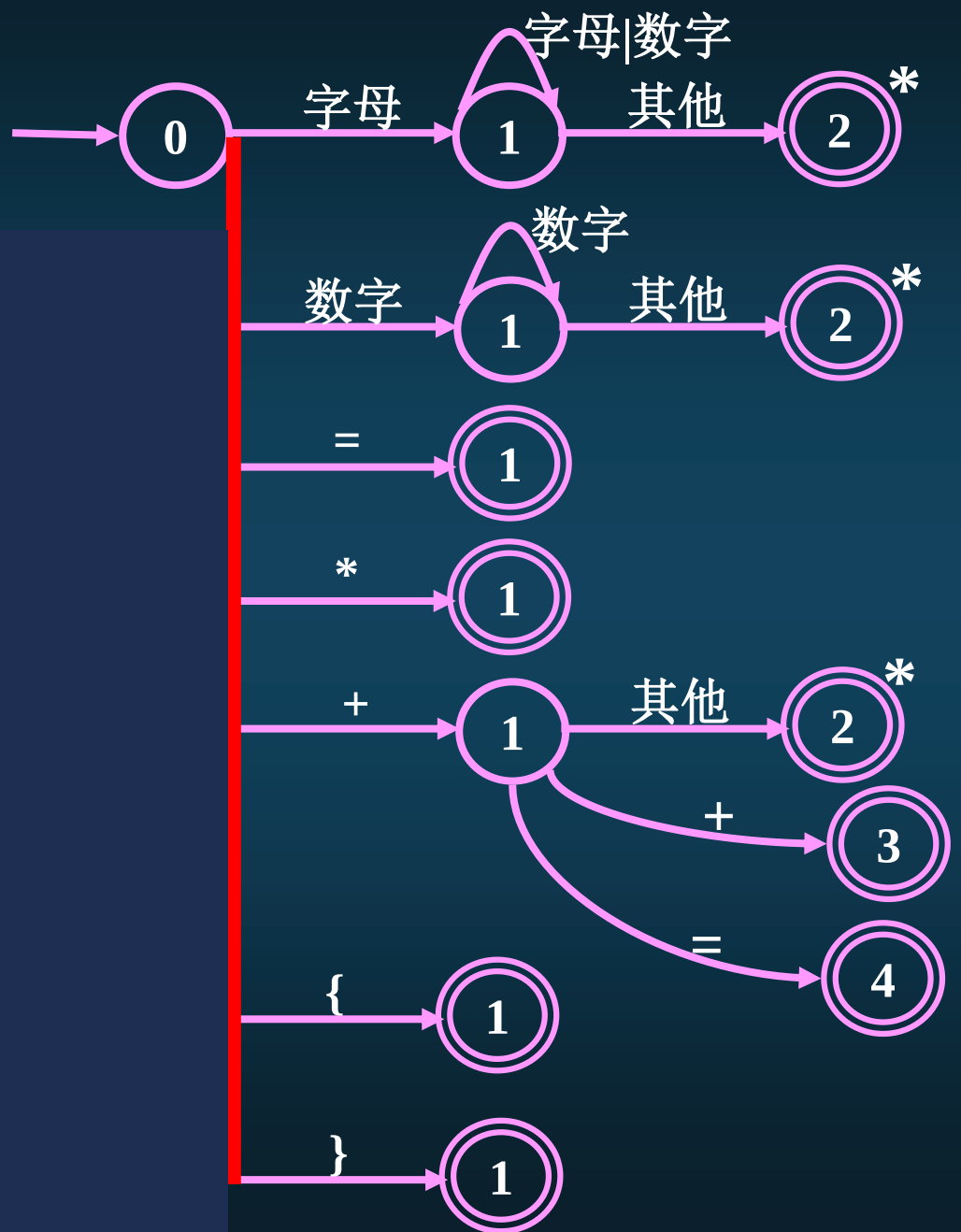
=:

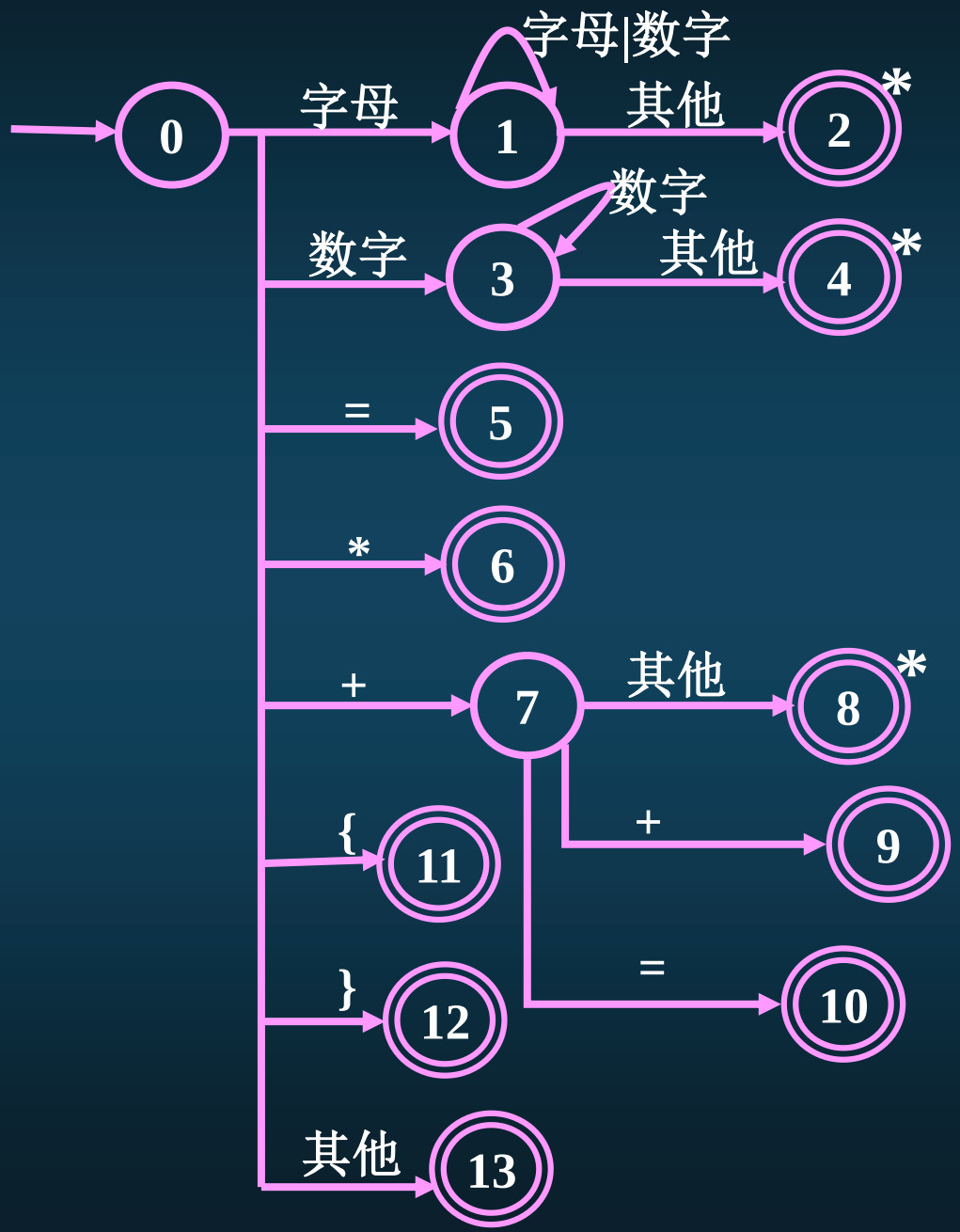
*:

+、++、+=:

{:

}:




字母|数字
字母
其他
*
0
1
2
数字
数字
其他
*
3
4
=
5
*
6
+
7
其他
*
8
+
9
{
11
=
10
}
12
其他
13

据语言词法规则

→ 状态转换图

→ 可行的扫描器

↓

数据中心法
程序中心法 } 存储和激活问题

- 状态转换图的实现之一 —— 数据中心法

将状态转换图看成一种数据结构(状态表)，用总控程序控制输入的源程序串在其上运行。

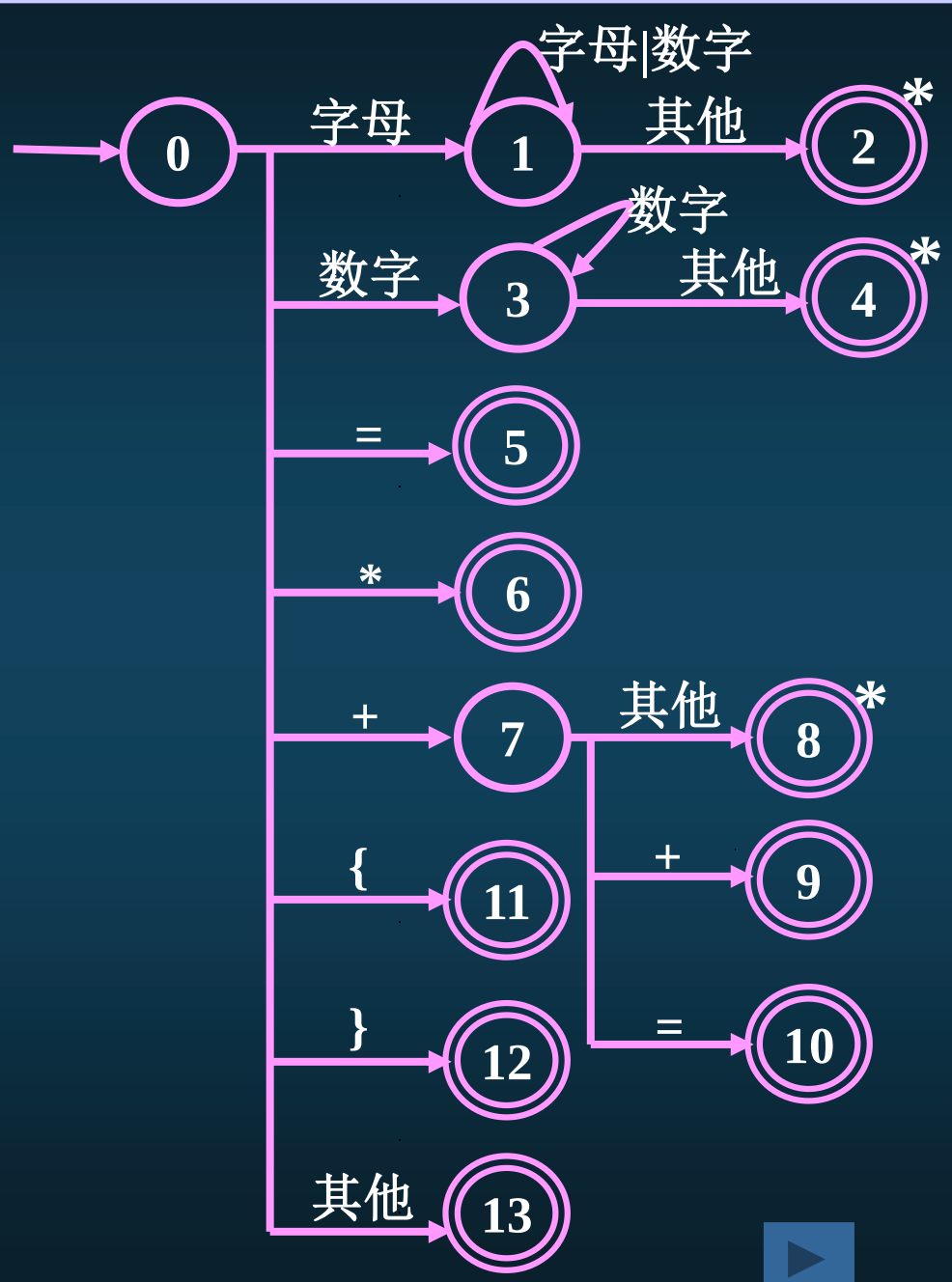
- 状态表 → 二级目录表

1. 主表： 数据项= 状态 + 分表地址或子程序入口

   状态 = 终态时，为子程序入口

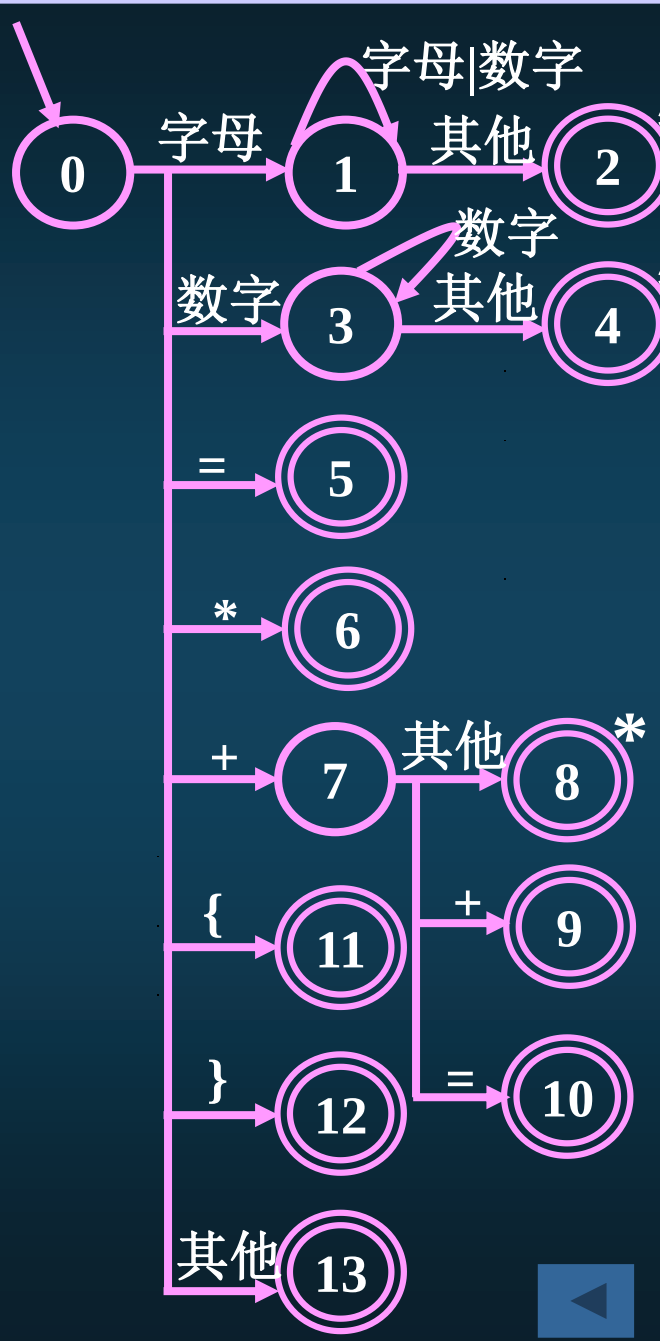
   状态 ≠ 终态时，为分表入口

2. 分表： 数据项= 当前输入字符 + 转换状态



主　表

状　态	分表地址
0	（1）
1	（2）
2*	（标识符保留字）
3	（3）
4*	子(整常数)
5*	子(=)
6*	子(+)
7	(4)
8*	子(+)
9 *	子(++)
10*	子(,)
11*	子({)
12*	子(})
13*	Error



分表 (1)

输入字符	转换状态
字母	1
数字	3
=	5*
*	6*
+	7
{	11
}	12
Other	13

分表 (2)

输入字符	转换状态
字母	1
数字	1
Other	2*

分表 (3)

输入字符	转换状态
数字	3
Other	4*

分表(4)

输入字符	转换状态
+	9*
=	10*
Other	8*

- **算法3.1(总控算法 — 二级目录表) :**

输入 : 字符串形式的源程序流 ;

输出: 源程序单词的属性字流;

算法:

/*B — 状态寄存器; W — 字符寄存器 ; W' — 字符串寄存器; */

(1) B =初态; W' = ' ';

(2)据B查主表 { 子程序入口, 转(3);
分表地址, 当前读入字符 → W ,指针前移;

组合单词W'=W'+W, 转(4);

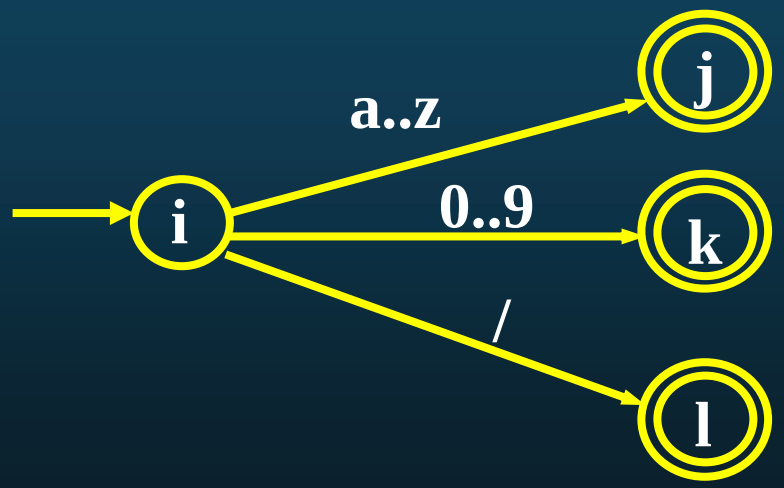
(3) 终态处理: 输出单词 (W')属性字, goto (1) ;

(4) 据W查分表, 将W对应的状态 → B, 转(2)。

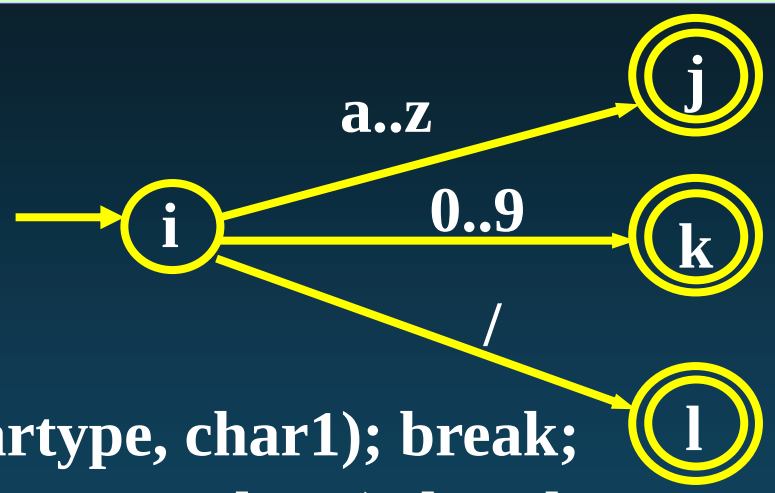
- 状态转换图的实现之二 —— 程序中心法

将状态转换图看成一个流程图，从初态开始对它的每个节点（状态）编写一函数或直接跟踪状态图从初态开始的转换完成所有分支的跟踪来编写程序。

例3.5 设单一小写字母或单一数字或“/”为合法单词，表示它们的状态转换图如图所示。



```
char char1;
  { char1=nextchar(   );
      if  (state=i )
          switch (char1)
           {
              case 'a'…'z': J (chartype, char1); break;
              case '0'…'9': K (chartype, char1); break;
              case ' / '     : L (chartype, char1); break;
             default: error;
           }
  }
```



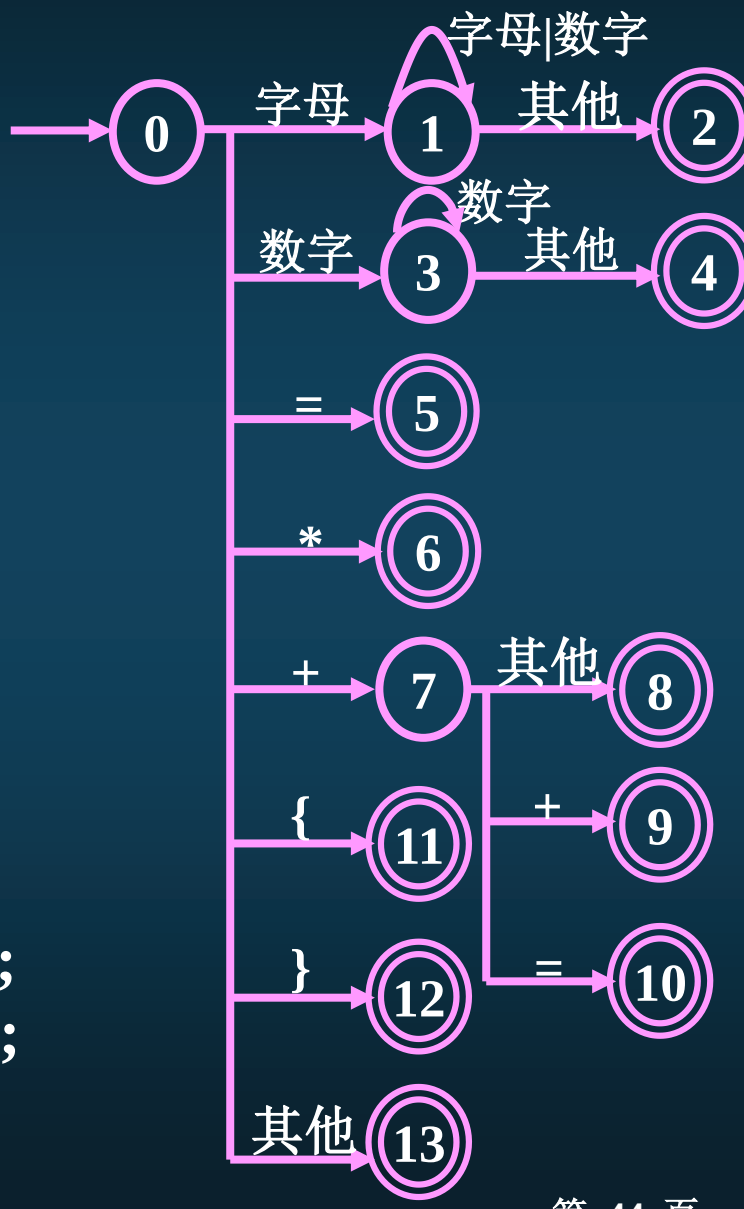
其中：

J，K，L为状态j，k，l所对应的函数；

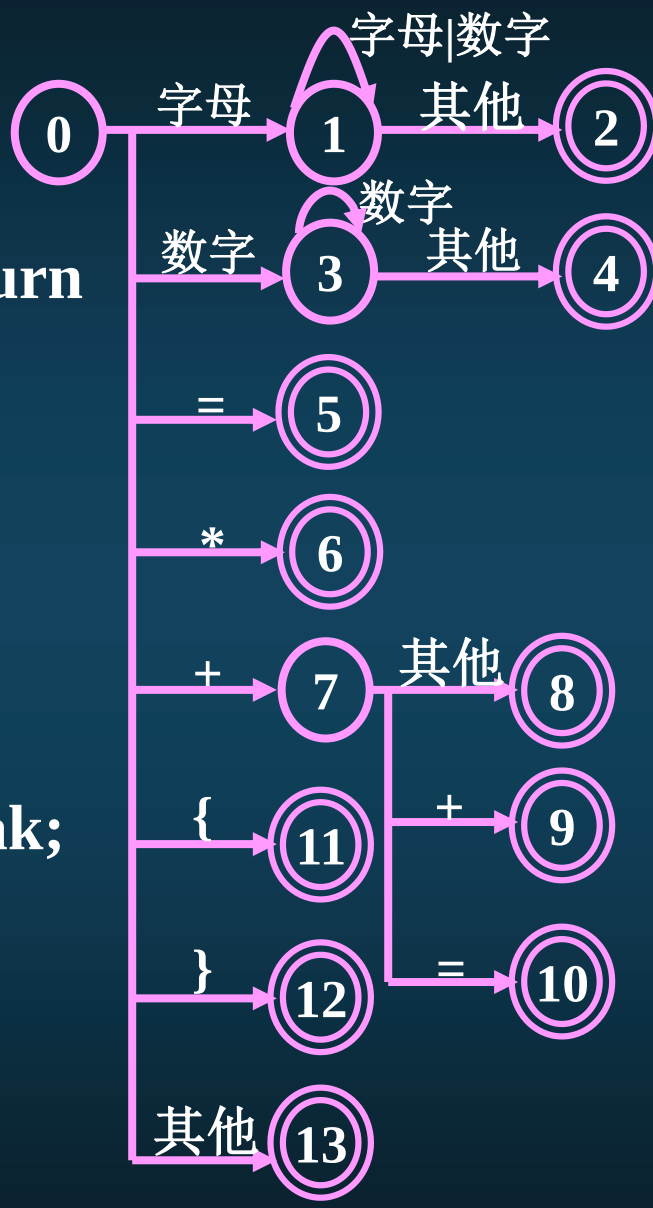
nextchar( )为一函数，功能是从当前扫描的源程序读取下一个字符；

```
int state = 0;
enum lettet ('a'..'z');
enum number ('0'..'9');
char char1;
while（1）
  {  char1 = nextchar( );
  switch(state)
    {
     case 0: switch (char1)
        {
          case  'a'…'z'   :  state = 1; break;
          case  '0'…'9'   :  state = 3; break;
          case  '='       :  state = 5; break;
          case  '*'       :  state = 6; break;
          case  '+'       :  state = 7; break;
          case  '{ '      :  state = 11; break;
          case  '} '      :  state = 12; break;
           default        :  state =13;
        }  break;
```

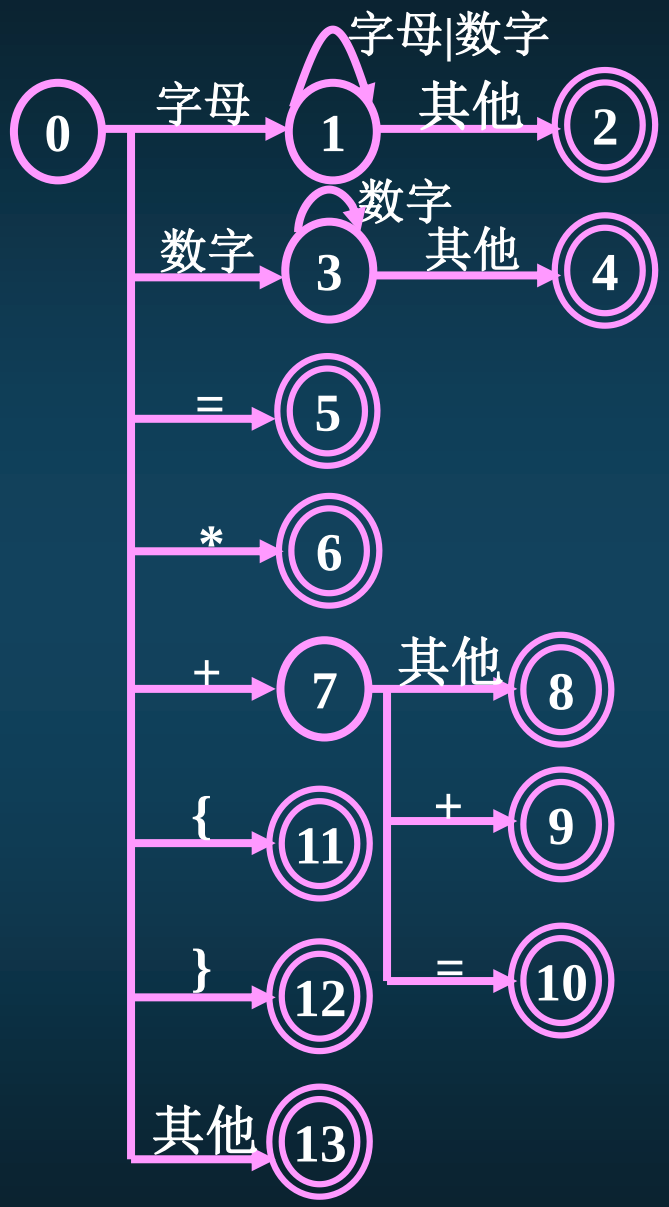
状态转换图的实现



```
case 1:  while (char1 == letter||number)
                char1 = nextchar();
            state = 2; break;
case 2:  untread();return (02,value) or return
            (01,value); state=0; break;
/* 函数untread()功能是回退一个已读进的字符;
   属性01表示关键字；属性02表示标识符 */
  case 3: while (char1 == number)
                char1 = nextchar();
                state = 4;  break;
  case 4: untread(); return (03 ,value); break;
/*  属性03表示无符号整常数   */
    case 5:  return (04,  ); break;
/*  属性04表示 “＝”   */
    case 6:  return (05,  ); break;
/*  属性05表示 “*”   */
```



```
case 7:  if (char1 == '+') state = 9;
         else  if (char1==”=”)  state = 10;
               else state=8; break;
case 8 :untread(); return(08, ); break;
/*  属性08表示 “+”  */
case  9:  return (09,  ); break;
/*  属性09表示 “++” */
case 10:  return (12,  ); break;
/*  属性12表示 “+=”  */
case 11:  return (10,  ); break;
/*  属性10表示 “{”  */
case 12:  return (11,  ); break;
/*  属性11表示 “}”  */
case 13 : error;
/*  error 是词法错误处理函数  */
}
}
```



第 3 章　词法分析 (Lexical Analysis)

3.1 词法分析程序的功能

3.2 词法分析器的设计与实现

3.3 词法分析器的自动生成

一．自动生成思想

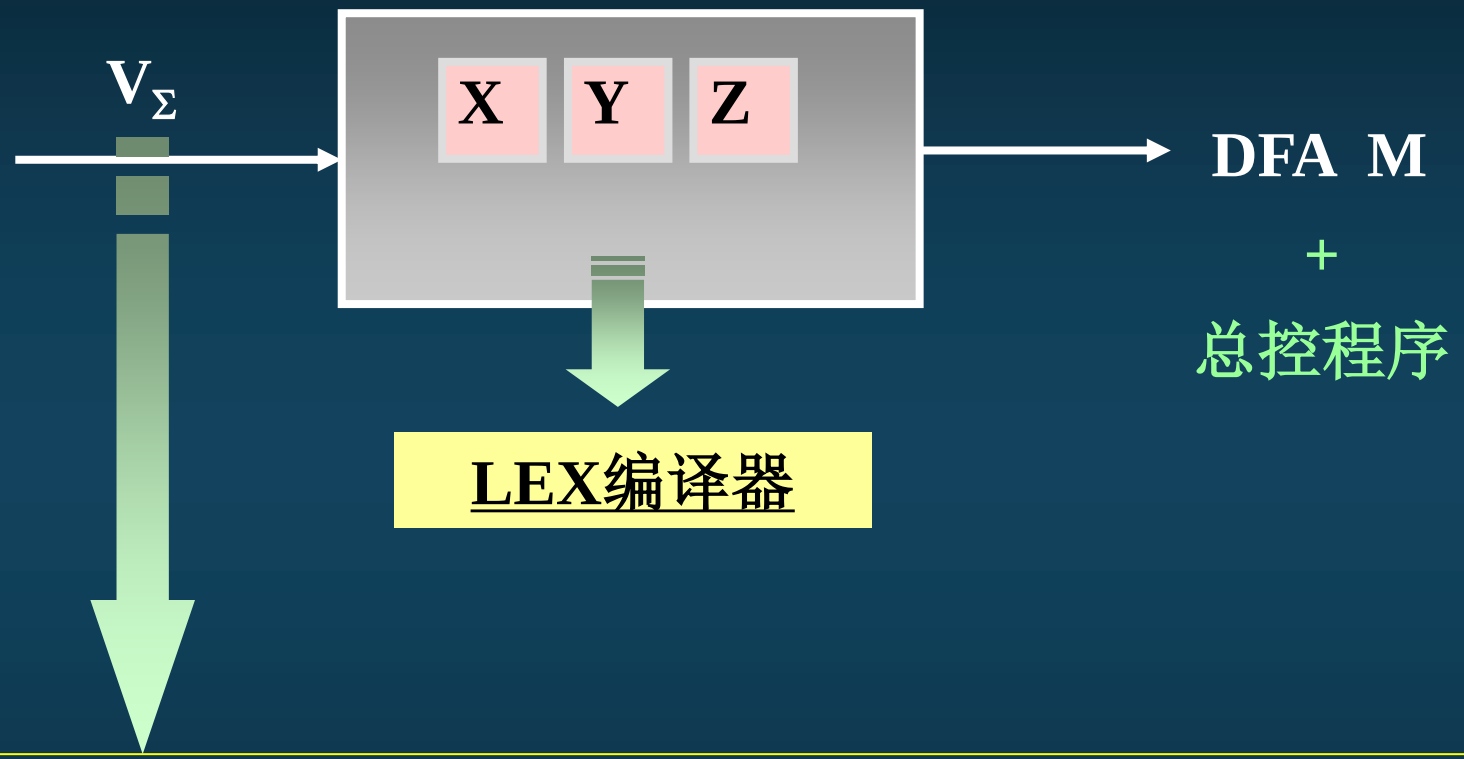
一个词法分析程序产生器接收用正规式表示的定义在某语言字母表Σ上的单词, 然后从此正规式出发

1）构造能识别正规式 描述的单词集(正规集)的非确定有限自动机NFA M’, 此步构造算法定义为X。

2）用子集法(子集法实现算法命名为Y)将M’ 确定化, 得到与其等价的DFA M；

3）用划分算法(命名为Z) 对M 化简, 得到DFA M"。

则这个DFA M "即是理论上的扫描器。



用正规式对语言的词法规则进行描述, 而正规式实际输入形式是用LEX语言来描述的。

- **算法3.2(总控算法)：**

输入：字符串形式的源程序流；

输出：源程序单词的属性字流；

算法：

/* I — 当前状态； a — 当前输入字符； A — 字符串寄存器； */

① I=0；A=‘空’；

② I如果是终态则转向 ⑤，否则向下；

③ 当前读入字符 =>a；指针前移；

④ I=I(a)；　　/* 送（I, a）的后继状态 => I */

A=A+a；　转②；　　/* “+”是指字符连接 */

⑤ 处理识别单词（A）。

二．扫描器自动生成器——LEX/FLEX

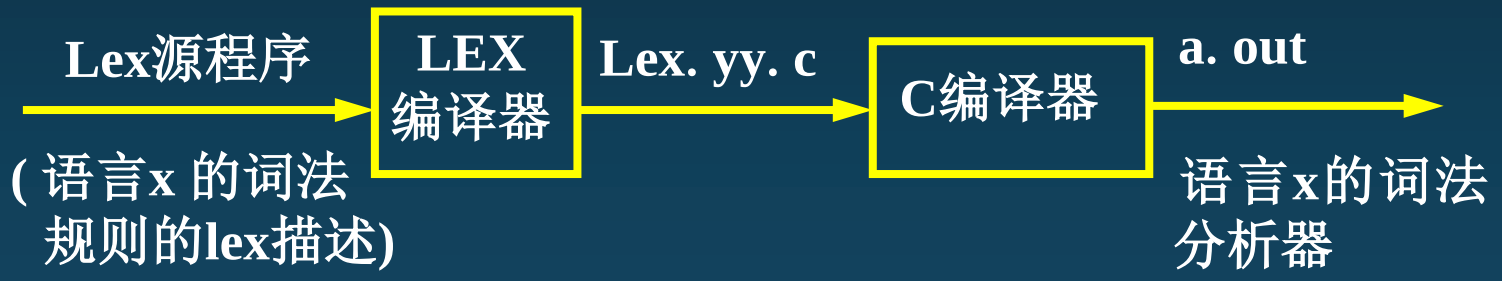
LEX ——（Lexical Analyzer Generator)

1972年贝尔实验室在UNIX上首先实现的，它是UNIX标准应用程序。

Flex(Fast Lexical Analyzer Generator)

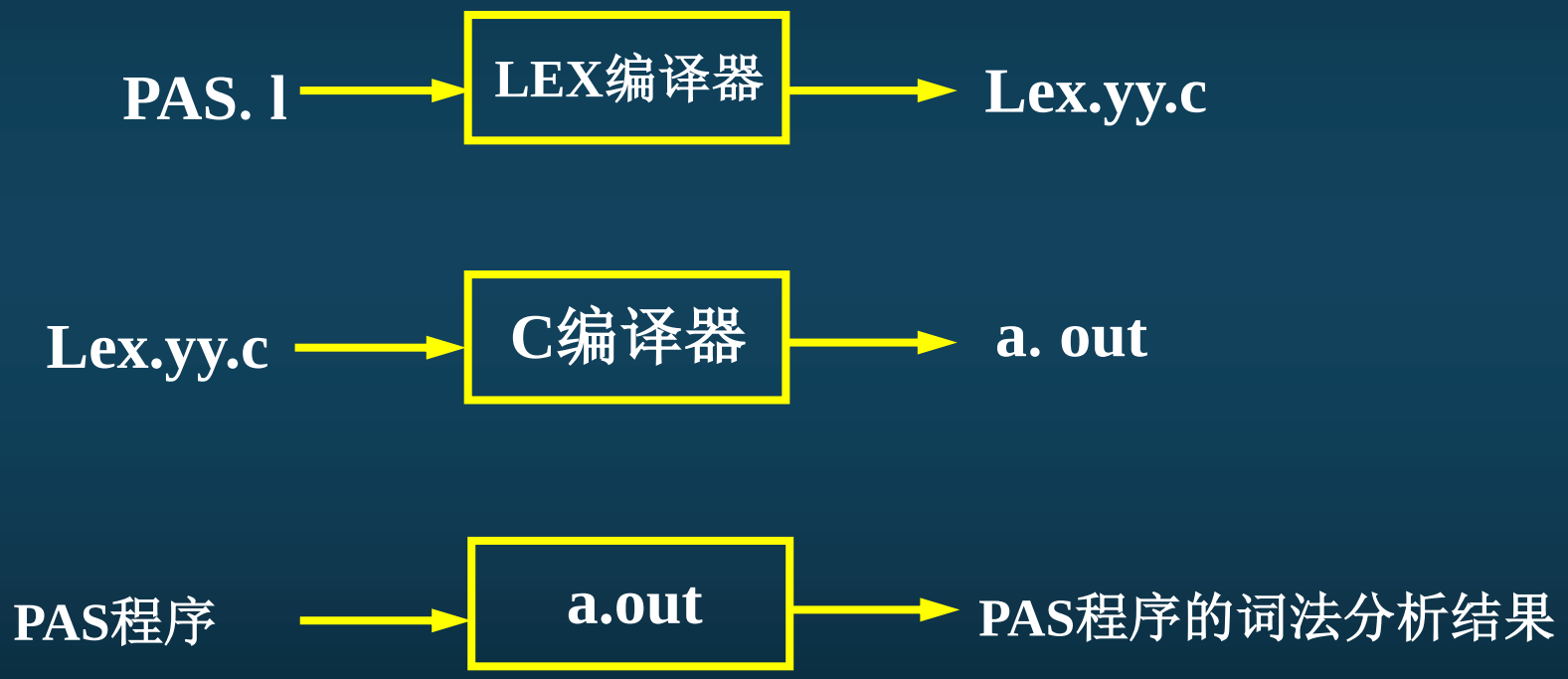
始于1984年的GNU工程推出，它是对LEX的扩充，同时也与LEX兼容。

LEX运行与应用过程



LEX编译器接收LEX源程序，由LEX编译器处理LEX源程序，产生一个词法分析器作为输出。在UNIX环境中，LEX编译器的输出是一个具有标准文件名lex. yy. c的C程序，经过C编译器的编译产生a. out文件，a. out是一个实际可以运行的词法分析器。

例3.6 :产生PAS语言的词法分析器



三个步骤：NO

（1） 编辑LEX源程序(例如，生成文本格式的PAS语言的LEX源文件PAS.l)；

（2）使用命令 **lex PAS.l** 运行LEX，正确则输出lex.yy.c；

（3）调用C编译器编译 lex.yy.c，并与其它C模块连接产生执行文件a.out 。

LEX程序结构

definitions　　Di　Ri

/* 包含模式宏定义和C语言的说明信息 */

说明部分

%%

rules　　Pi　{ Ai }

规则部分

%%

subroutines

/* 规则动作部分所需的辅助过程的的C代码 */

用户代码部分

例如，有C说明和LEX宏定义示例如下

```
%{
# include <stdio.h>
# include <ctype.h>
extern  int flag
# define  Marry  1
# define  Lida   2
int ERROR= -1
char  c
%}
digit   [0-9]
alpha  [a-zA-Z]
alnum  [a-zA-Z0-9]
%%
```

C说明

正规式辅助定义

说明部分

识别规则部分一般形式是　Pi　{ Ai }

模式1	动作1
模式2	动作2
……	
模式n	动作n

Pi　{ Ai }

模 式	说　明	示　例
x	匹配单个字符x	
[abc]	匹配a或b或c	[a-h0-5]
[^abcde]	匹配除去a-e之间的任意字符(可为[^a-e])	[^abA-Z\n]表示匹配除小写字母a,b大写字母A－Z和换行符外任意字符
\	转义符定义同ANSI C	
•	匹配除去换行符之外的任意字符	
r*	r是正规式，r*匹配0个或多个r	
r+	r是正规式，r+匹配1个或多个r	
r?	r是正规式，r? 匹配0个或1个r	
	……	

例3.7　下表给出了某语言的单词的词法分析结果。

单词的正规式表示	属　性	单 词 值
ws		
if	if	
then	then	
else	else	
id	id	名表指针
num	num	常数表指针
<	relop	LT
<=	relop	LE
=	relop	EQ
<>	relop	NE
>	relop	GT
>=	relop	GE

该语言的LEX源程序如下:

```
%{
# include <stdio.h>
# include <ctype.h>
# include <string.h>
# define  IF          1
# define  THEN        2
# define  ELSE        3
# define  ID          4
# define  NUMBER      5
# define  LT          6
# define  LE          7
# define  EQ          8
# define  NE          9
# define  GT          10
# define  GE          11
%}
```

```
/* 正规式模式宏定义 */
        digit       [0-9]
        alpha       [a-zA-Z]
        alnum       [a-zA-Z0-9]
        delim       [ \t\n]
        ws          {delim}+
        id          {letter}({letter}|{digit})*
        number      {digit}+(\.{digit}+)?(E[+|-]?{digit}+)?
%%
```

```
{ws}        {/* no action and no return */}
if          {return(IF);}
then        {return(THEN);}
else        {return(ELSE);}
{id}        {yylval = install_id(); return(ID);}
{number}    {yylval=install_num(); return(NUMBER);}
"<"         {yylval = LT; return(RELOP);}
"<="        {yylval = LE; return(RELOP);}
"="         {yylval = EQ; return(RELOP);}
"<>"        {yylval = NE; return(RELOP);}
">"         {yylval = GT; return(RELOP);}
">="        {yylval = GE; return(RELOP);}
%%
```

```
install_id()
{
/* procedure to install the lexeme, whose first
   character is printed to by yytext and  whose
   length is yyleng, into the symbol table and
  return a pointer thereto */
}
install_num()
 {
   /* simllar procedure to install alexeme that is a number */
     }
```

词法分析器产生器的实现

(1) 对LEX源程序识别规则中的每个P_i构造一个相应的NFA M_i。

(2) 引入惟一初态X，从初态X通过ε弧将所有NFA M_i (i=1，..., n) 连接成新的NFA M'。(1), (2)两步实际是完成从正规式到非确定有限自动机的构造，其算法为X。

(3) 对NFA M '确定化，产生DFA M，实现此步变换的算法为Y。

(4) DFA M化简，其算法为Z。

(5) 给出总控程序。

- **算法3.2(总控算法)：**

输入：字符串形式的源程序流；

输出：源程序单词的属性字流；

算法：

/* I — 当前状态； a — 当前输入字符； A — 字符串寄存器； */

① I=0；A=‘空’；

② I如果是终态则转向 ⑤，否则向下；

③ 当前读入字符 =>a；

④ I=I(a)；　　/* 送（I, a）的后继状态 => I */

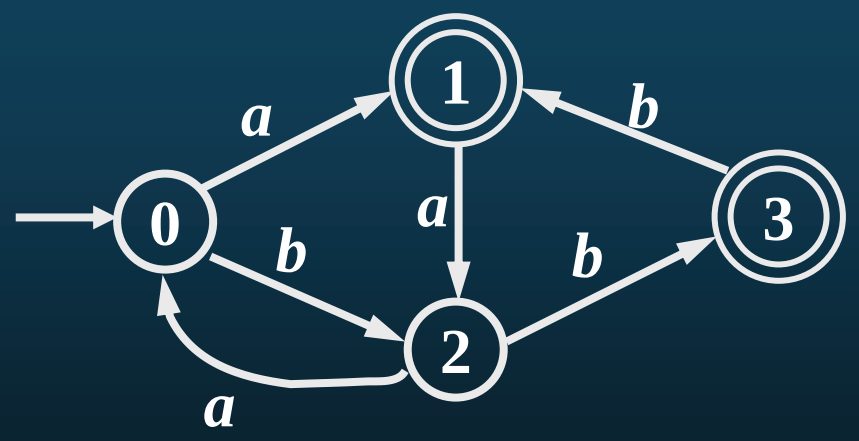
A=A+a；　转②　　/* “+”是指字符连接 */

⑤ 处理识别单词（A）。

- **算法3.2的问题：** 单词识别的二义问题。

该算法不能识别单词的最长子串匹配，即当遇到DFA的终态时，应继续前进，直到无法前进为止（即当前状态与当前输入字符序偶的转换无后继状态）。

例如，设有DFA如下。试识别输入串*aabaabbbb*。



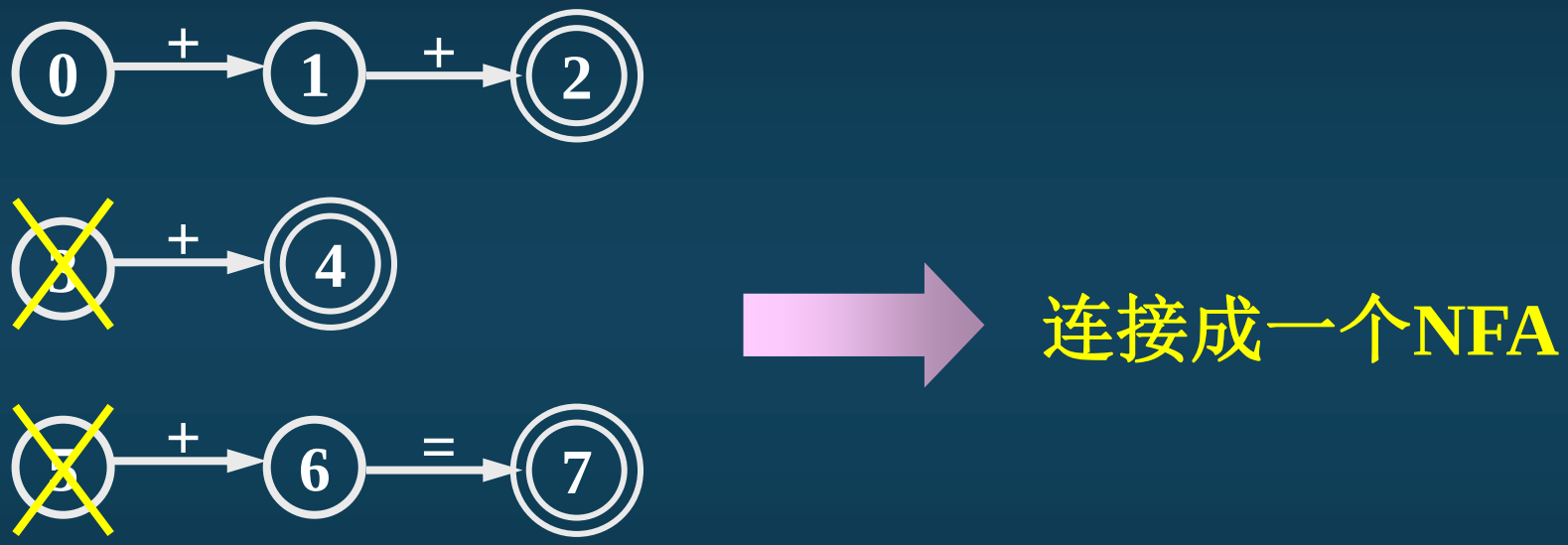
按所给总控算法识别：

a , *a* , *baa* , *bb* , *bb* .

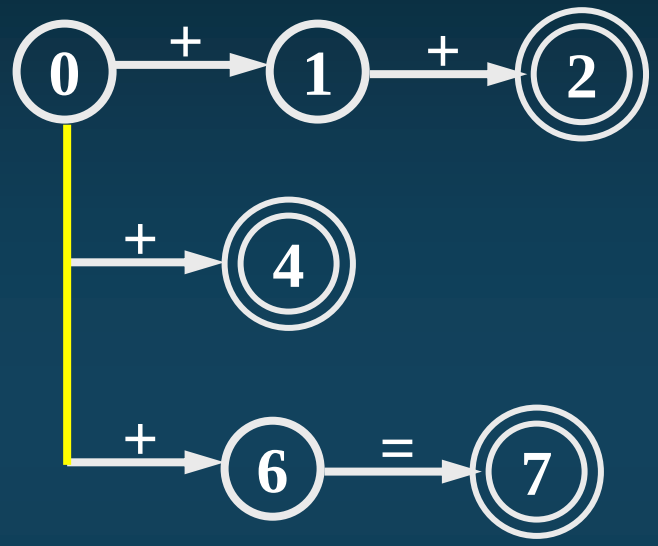
按最长匹配识别：

aab , *aabb* , *bb* .

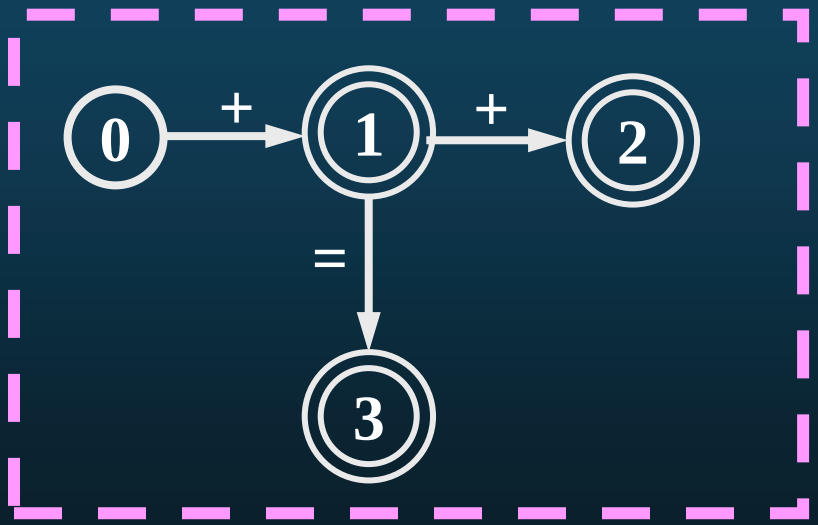
例如，设有识别C语言“+”“++”“+=”的NFA如下。



例如，设有识别C语言“+”“++”“+=”的NFA如下。



		+	=
0	{ 0 }	{1,4,6 }	
1	{1,4,6 }	{2}	{ 7 }
2	{ 2 }		
3	{ 7 }		



	+	=
0	1	
1*	2	3
2*		
3*		

LEX中的解决方法——考虑优先匹配：

一个字符串有两条规则可以同时匹配，按照优先级处理。　如：　**int1 和 a+++b 。**

➢ 词法分析器构造要素

处理对象 —— 字符串形式的源程序

处理依据 —— 源语言词法规则

处理结果 —— 属性字表示的单词(单词属性、内码)

词法分析器核心功能：输入扫描、识别、输出映射处理。

第三章提要

- ✓词法分析的任务和词法分析方法。
- ✓ 词法分析程序总体设计的考虑:

  词法分析程序输入对象的分析、提炼及相应输出属性字的设计。
- ✓词法分析程序的设计工具与实现机制。
- ✓词法分析程序自动生成的思想。
- ✓LEX应用与LEX源程序的结构与描述。

end

(1) 设有C语句的程序段如下：

```
while ( i && ++j )
      { c=2.19;
        j+=k;
        i++;
        }
```

则经过词法分析后可以识别的单词个数是__B__个。

A) 19　　B) 20　　C) 21　　D) 23

(2) 下面____B____不是预处理程序完成的功能。

A) 滤掉源程序中的注释　B) 查找源程序中无用字符

C) 进行宏替换　D) 实现文件包含的嵌入和条件编译的嵌

(3) 识别各类单词的FA(状态转换图)合并后得到的FA__A__。

A) 可能是NFA也可能是DFA　B) 一定是DFA

C) 一定是NFA　D) 是最小的DFA

判断正误。

(1) Lex是典型的词法分析程序。 ×

(2) 词法分析的依据是源语言的文法规则。×

(3) 源程序中的单词是具有独立意义的短语。×

(4) 单词的属性字一般应该包括单词类别和单词内码。 √

(5) 编译的预处理程序的处理对象是源程序。√